Full Stack React Apps (with Next.js)

Addison Bartelli and Hunter Long

React Overview

- **React.js** is a front-end JavaScript/TypeScript library for building user interfaces
- Allows user to create modular **components**, bits of HTML code that can be instantiated across a project
- These components can have various **properties** for flexibility
- **States** and **Effects** allow them to respond to new information and external data

What is a Component?

- The building blocks of React
- JavaScript/TypeScript function that exports an HTML element
- Can be **instantiated** across project

```
export default function Profile() {
  return (
    <img
          src="https://react.dev/im
          ages/docs/scientists/MK3e
          W3Am.jpg"
          alt="Katherine Johnson"
    />
  )
}
```

```
export default function Gallery() {
  return (
    <section>
      <h1>Amazing scientists</h1>
      <Profile />
      <Profile />
      <Profile />
    </section>
  );
}
```

JSX

- The notation used by React apps to embed write HTML in JavaScript
- A sort of “HTML literal”
- JavaScript expressions themselves can be embedded in JSX

```
return (
        <section className="post border-2">
            <a href={props.pdata.link}>{props.pdata.link}</a>
               <span onClick={deleteHandle} id={props.pdata.id.toString()}
               className="cursor-pointer underline">Delete</span>
        </section>
    );
}
```

Properties

- Components can take input and set **properties**
- Allows the same component to represent different things
  - Ex. the “Profile” component showing different people by having “name” and “image” properties

```
function Avatar({ name, imageId }) {
  return (
    <img
      className="avatar"
      src={imageId}
      alt={name}
      width=100
      height=100
    />
  );
}
```

```
export default function Profile() {
  return (
    <div>
      <Avatar
        name={ 'Katherine Johnson' }
        imageId={ 'YfeOqp2' }
      />
      <Avatar
        name={ 'Aklilu Lemma' }
        imageId={ 'OKS67lh' }
      />
    </div>
  );
}
```

Using Components with Iterables

```
export default function Profile({ name }) {
	return (
		<div>
			<h1>{name}</h1>
		</div>
	)
}
```

```
namesList = {
	'Katherine Johnson',
	'Aklilu Lemma',
	'Grace Hopper'
}

return (
	<div>
		{names.map((name) => (
			<Profile name={name} />
		))}
	</div>
)
```

What is State?

- **State variables** are the “memory” of a component
- Necessary because in React, regular variables aren’t enough
  - Memory doesn’t persist between renders (needed!)
  - Changing regular variables doesn’t trigger re-renders
- These are important to make elements respond dynamically!

```
const [myNum, setMyNum] = useState(0);

setMyNum(myNum + 1);
```

Implementing State with useState

- **useState** is a hook providing:
  - A **state variable** to retain data between renders
  - A **state setter function** to modify that data
- useState must be imported: `import { useState } from 'react';`

```
const [myNum, setMyNum] = useState(0);

setMyNum(myNum + 1);
```

Documentation: link

Act on state with useEffect

- **Effect:** a side effect caused specifically by rendering itself
  - Setting up a server connection **is** an Effect
  - A button's onClick function is **not** an Effect
- **useEffect** is another hook react provides
  - Allows component to be **synchronized with external system**
- Must be imported: `import { useEffect } from 'react';`

Documentation: link

useEffect Example

```
import { useState, useEffect } from 'react';
import { createConnection } from './chat.js';

function ChatRoom({ roomId }) {
    const [serverUrl, setServerUrl] = useState('https://localhost:1234');

    useEffect(() => {
        const connection = createConnection(serverUrl, roomId);
        connection.connect();
        return () => {
            connection.disconnect();
        };
    }, [serverUrl, roomId]);
    // ...
}
```

setup code

cleanup code

dependencies

Documentation: link

Client vs Server

Client

- Access elements on the page (textboxes, buttons)
- Receives input (keystrokes, camera, microphone)
- Store cookies, data in localStorage
- Read certain browser properties

Server

- Accesses the database
- Authenticates users
- Any logic that must not be tampered with

Both

- Render HTML Elements
- Access APIs not taking secret data

What is a Full-Stack Application?

A traditional network-enabled application runs entirely on the client (file explorer, web browser, microsoft word) or entirely as a server (mail server, file server).

A “full-stack” application is really 2+ applications: on the server and the client. We just think of them as one application for convenience and as an abstraction.

“Full-stack” often invokes “website”, but any both-sides network app counts.

Examples:

- **Gmail** - A mail server and webserver exists on the “back” end; An HTML and JavaScript website exists on the “front” end.
- **Multiplayer Minecraft** - A game client application runs on your computer in the “front” end; An application responsible for maintaining state (e.g. player positions, inventories) runs on the server in the “back” end.

Next.js Overview

"Next.js is a React framework for building full-stack web applications."
- nextjs.org/docs

React alone is just a library for designing UI and managing client-side state.

Frameworks expand upon react with a bunch of tools
- Automatic routing (name/page.tsx -> name.html)
- Defining custom HTTP routes (useful for making APIs!)
- Development server
- General optimizations
- Server-side rendering / server components
  - (This is the one we care about today!!!)

To start with Next.js:

```
addie@reality  ~  npx create-next-app@latest
```

Server-Client Divide in Next.js

```
// Server Side HomePage
export default async function ServerHome() {
    ensurePostsTable();
      You, 11 minutes ago • Uncommitted changes
    let posts: ServerCreatedPost[] = [];
    try {
        posts = db.prepare("SELECT * FROM Posts").all()
    } catch {
        console.error("SQL error in get /posts");
    }

    return (
        <ClientHome posts={posts}></ClientHome>
    )
}
```

```
1    "use client"
// This is a client component
export default function ClientHome( {posts: postsProp} : {posts: ServerCreatedPost[]} ) {

  /* We set up some states here that the application can "react" to */

  const [posts, setPosts] = useState(postsProp);
```

```
switch (mode) {
  case "dark":
    body.classList.remove("light");
    body.classList.add("dark");
    break;
```

```
return (
  <main className='flex h-full w-full flex-row p-8 box-border'>
    <div className="h-full w-2/3 flex flex-col gap-y-4">
      <MakePost addLocalPost={addLocalPost}></MakePost>
      <button onClick={refreshPosts} className="w-fit">Refresh Posts</
      <DarkModeRadio updateColor={updateColor} />
    </div>
    <div className="grow h-full flex flex-col gap-y-4 mx-8 overflow-sc
      {posts.map(p => <Post pdata={p} removeLocalPost={removeLocalPost
    </div>
  </main>
)
```

Let's Take a Look at it All in Action...

🎉 Thank You 🎉

Any Questions?

Reach out to one of us on the HackKU Discord!

Find the demo at github.com/addiebart/hackku26-react-demo